

Article

Container Allocation in Cloud Environment Using Multi-Agent Deep Reinforcement Learning

Tom Danino ^{1,†} , Yehuda Ben-Shimol ^{1,*,†}  and Shlomo Greenberg ^{1,2,*,†} ¹ School of Electrical and Computer Engineering, Ben Gurion University, Beer Sheva 84105, Israel² Department of Computer Science, Sami Shamoon College of Engineering, Beer Sheva 84100, Israel

* Correspondence: benshimo@bgu.ac.il (Y.B.-S.); shlomo.greenberg@gmail.com (S.G.)

† All authors contributed equally to this work.

Abstract: Nowadays, many computation tasks are carried out using cloud computing services and virtualization technology. The intensive resource requirements of virtual machines have led to the adoption of a lighter solution based on containers. Containers isolate packaged applications and their dependencies, and they can also operate as part of distributed applications. Containers can be distributed over a cluster of computers with available resources, such as the CPU, memory, and communication bandwidth. Any container distribution mechanism should consider resource availability and their impact on overall performance. This work suggests a new approach to assigning containers to servers in the cloud, while meeting computing and communication resource requirements and minimizing the overall task completion time. We introduce a multi-agent environment using a deep reinforcement learning-based decision mechanism. The high action space complexity is tackled by decentralizing the allocation decisions among multiple agents. Considering the interactions among the agents, we introduce a new cooperative mechanism for a state and reward design, resulting in efficient container assignments. The performances of both long short term memory (LSTM) and memory augmented-based agents are examined, for solving the challenging container assignment problem. Experimental results demonstrated an improvement of up to 28% in the execution runtime compared to existing bin-packing heuristics and the common Kubernetes industrial tool.

Keywords: cloud computing; Deep-RL; multi-agent; actor-critic; Kubernetes



Citation: Danino, T.; Ben-Shimol, Y.; Greenberg, S. Container Allocation in Cloud Environment Using Multi-Agent Deep Reinforcement Learning. *Electronics* **2023**, *12*, 2614. <https://doi.org/10.3390/electronics12122614>

Academic Editors: Neng-Chung Wang, Tien-Wen Sung and Chao-Yang Lee

Received: 19 April 2023

Revised: 6 June 2023

Accepted: 7 June 2023

Published: 9 June 2023



Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Recent years have shown cloud computing to be an important solution for large-scale massive data processing, as offered by major computing service providers. Much of the wide adoption of cloud computing as an on-demand service platform is made possible due to the success of virtualization technology and the usage of virtual machines (VMs) and containers. Both containers and VMs are packaged computing environments that combine various IT components that are isolated from the rest of the hosting system. A container is a standard software unit that packages up code and all its libraries and dependencies, so that the application runs quickly and reliably in various cloud computing environments.

The use of containerized applications significantly reduces the required memory resources compared to running the same application on a VM [1]. Typically, containers are designed to be activated and deactivated during their life cycle and can be redeployed in the same manner, regardless of the infrastructure [2]. Using containers enables multiple applications to share the same OS, i.e., running on shared resources on the same machine. Any container orchestration, such as the common Kubernetes [3], should dynamically and efficiently distribute the containers to available servers. In the case of a distributed application, containers also require networking resources in addition to local CPU and memory resources. Moreover, the application code may need to be executed using more than one container. Therefore, the efficient distribution of containers needs to consider

the availability of both local resources (i.e., server resources), and global resources (e.g., network resources). A geo-distributed deployment that relies on Kubernetes and extends it with self-adaptation and network-aware placement capabilities is presented in [4].

Reinforcement learning (RL) has been recently adopted to solve cloud and edge-computing resource allocation problems [4–9], and specifically container placement [10,11]. Busoniu et al. [12] presented a comprehensive survey of multi-agents where the agents are capable of discovering a solution on their own using reinforcement learning. A reinforcement learning agent learns by trial-and-error interaction with its dynamic environment. The agent perceives the complete state of the environment and takes an action which causes a transition to a new state. The agent receives reward feedback that evaluates the quality of this transition. Alwarafy et al. presented a multi-agent deep reinforcement learning (DRL)-based framework for dynamic radio resource allocation [13]. Horovitz and Arian [14] presented a new method for improving reinforcement Q-Learning auto-scaling with faster convergence and reduced state and action space in a distributed cloud environment.

Finding an efficient container placement scheme to minimize the overall execution time is known as an NP-hard combinatorial optimization problem [11]. The complexity increases exponentially as the number of containers increases [15]. The complexity of assigning containers to available servers is frequently derived by fitting the cloud environment and constraints into a known (NP-complete) problem model. The most commonly used reductions are bin packing [15–17] and integer linear programming [4,18].

Usmani et al. [19] suggested modeling the problem of resource allocation, while minimizing the number of physical machines using a bin-packing algorithm [19]. Zhang et al. [16] propose a novel container placement strategy based on a bin packing heuristic that simultaneously considers both virtual and physical machines. Abrishami et al. [20] adopted a two-phase scheduling algorithm for computer grids, which aimed to minimize the cost of workflow execution, while meeting user-defined constraints for the cloud environment. Li and Cai [21] proposed a heuristic approach to elastic virtual machine provisioning, demonstrating a decrease in virtual machine rental costs of about 78%. Cai et al. [22] proposed a unit-aware rule-based heuristic (URH), which distributes the workflow deadline to competitive task units, allowing the use of appropriate time slots on the rented virtual machines, minimizing the VM rental cost. Chen et al. [23] presented an entropy based stochastic workload scheduler, to allocate clients' tasks to different cloud data centers. They proposed a QoS model to measure the overall performance. Experimental results demonstrated an improvement of the accumulative QoS and sojourn time by up to 56.1% and 25.4%, respectively, compared to a baseline greedy algorithm.

Recurrent artificial neural networks are remarkably adept at sequence and reinforcement learning but are limited in their ability to represent complex data structures and to store data over long timescales, owing to a lack of external memory [24]. Graves et al. [24] introduced a machine learning model called a differentiable neural computer (DNC), which consists of a neural network coupled to an external memory matrix. They demonstrated that a DNC-based network has the ability to represent complex data structures and to learn and store sequential data over long timescales. We adapted the reinforcement-based DNC model proposed by [24], due to its proven ability to represent variables and complex data structures and to store data over long timescales. In this work, we examine the benefit of using a DNC-based model applied to container allocation.

This paper presents a multi-agent based solution using a Deep RL approach to deal with the challenges of the container allocation problem. We consider and evaluate two different types of agent: LSTM-based agents and DNC-based agents. We propose an DRL-based multi-agent framework to optimize a shared objective, i.e., the efficient placement of containers, using a set of agents in a mixed cooperative–competitive environment. Each agent is provided with a shaped reward and may collaborate with other agents to improve container allocation strategies.

We compared the performance of the LSTM and the DNC multi-agents against the well-known bin packing heuristics and the common Kubernetes allocation mechanism.

Experimental results showed that both of the DRL-based approaches were superior in terms of the overall runtime and demonstrated an improvement of about 28% compared to the existing techniques.

This work proposes an efficient model-free multi-agent-based approach that enables the collaboration of multiple agents to solve the container allocation problem in a real cloud environment. The proposed method uses a deep reinforcement learning approach, which is efficiently applied to the challenging container placement problem.

In addition, other contributions of this paper are listed below:

- We present an integrated solution for the proposed container placement approach with the well-known Kubernetes orchestration tool;
- We extensively evaluate the proposed approach in a real cloud environment, demonstrating the superiority of the proposed multi-agent model-free RL-based solution concerning other placement control policies;
- We compare the performance of LSTM-based and DNC memory augmented-based agents against well-known bin packing heuristics algorithms and Kubernetes tool, while applying them to resource allocation problems in a containerized environment.

The rest of this paper is organized as follows: Section 2 reviews related work. Section 3 formulates the allocation problem, while section 4 presents the proposed approach. Section 5 introduces the suggested container allocation framework. Section 6 shows the experimental results, and Section 7 concludes the paper.

2. Related Work

This section introduces existing related research works presenting different placement approaches for cloud applications subject to varying workloads. We mainly focus on Deep RL methods successfully used to solve a wide range of resource allocation problems. The RL-based approach represents an interesting approach to the runtime self-management of cloud systems [5]. Reinforcement learning has been mostly applied to devise policies for VM allocation and provisioning and to manage containers [4–6,10,11,14], solving the cloud resource allocation problem.

Rossi et al. [4] proposed a model-based reinforcement learning approach to control the number of replicas of individual containers based on the application response time. They also proposed an effective RL-based approach, to cope with varying workloads [5]. In other work, they demonstrated the benefits and flexibility of the model-based reinforcement learning policy compared to the common scaling policy of Kubernetes [10].

Liu et al. [6] suggested a hierarchical framework of cloud resource allocation and power management using LSTM neural networks and deep reinforcement learning. They showed up to a 60% energy saving compared to the round-robin heuristics approach, at the cost of a larger latency. Yuan et al. [11] presented a DRL-based container placement scheme to place container instances on servers, while considering end-to-end delay and resource utilization costs. The proposed method is based on an actor–critic approach and outperformed other DRL methods by up to 28%, in terms of reducing the delay and deployment costs.

Nasir et al. [25] proposed a model-free multi-agent DRL-based framework for dynamic power allocation in wireless networks. The proposed method learns a policy that guides all links to adjust power levels under practical constraints, such as delayed information exchange and incomplete cross-link. A centralized network trainer gathers local observations from all network agents. This approach is computationally efficient and robust.

In contrast to previous works that used a centralized approach for resource allocation in containerized environments [5,10,11], the present research adopts a decentralized approach to the container assignment problem. We propose a multi-agent framework to optimize the placement of the containers using a set of agents in a mixed cooperative–competitive environment. Moreover, we suggest an DRL-based model-free approach, while [4,5,10] proposed a model-based RL approach. To the best of our knowledge, this

work is the first to adopt a DNC memory augmented-based model to solve the complex container assignment problem.

3. Problem Formulation

Figure 1 depicts a typical cloud environment with M servers, each characterized by its available resources (i.e., CPU, memory, and bandwidth).

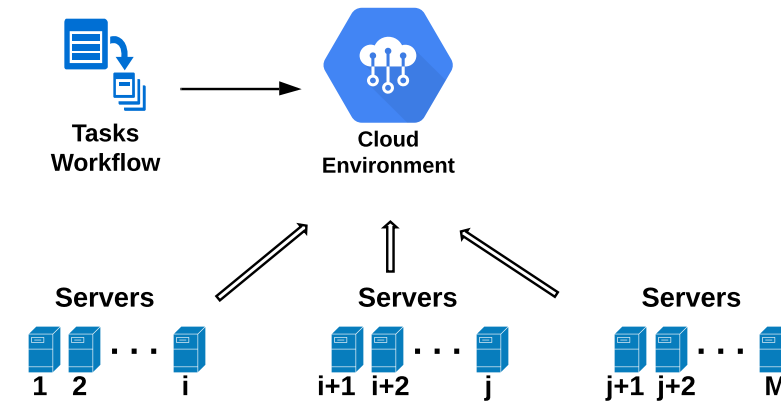


Figure 1. General cloud environment architecture.

The container placement problem can be considered as batch assigning, where a batch is composed of N tasks, each with different local resource requirements for CPU computing power and memory. Some of the tasks communicate with other tasks, running on different servers, and therefore require extra bandwidth.

All the tasks in the batch should be efficiently and simultaneously assigned to the cloud servers, to fulfill the following criteria: (a) the total runtime of the entire batch should be minimized, and (b) the resource requirements of all tasks in the batch are fully met.

Let us assume a batch with K tasks, i.e., K containers, and a set of M available servers $S_1, \dots, S_M$. The required memory and computation power for each task is given as follows:

$$\mathbf{m} = (m_1, m_2, \dots, m_k) \quad (1)$$

$$\mathbf{c} = (c_1, c_2, \dots, c_K) \quad (2)$$

where m_i and c_i , represent the memory and computation power required by the i th container. The bandwidth required for inter-task communication is given by

$$\mathbf{B} = \begin{pmatrix} b_{11} & \dots & b_{1K} \\ \vdots & \ddots & \vdots \\ b_{K1} & \dots & b_{KK} \end{pmatrix} \quad (3)$$

where $b_{i,j}$ represents the bandwidth required for communication from task i to task j . The total required bandwidth is given by

$$\mathbf{b} = \mathbf{B} \cdot \mathbf{e}^T \quad (4)$$

where $\mathbf{e}$ is a K “ones” row vector.

The servers resources are represented by the following vectors:

$$\mathbf{S}\mathbf{m} = (s_{m_1}, s_{m_2}, \dots, s_{m_M}) \quad (5)$$

$$\mathbf{S}\mathbf{c} = (s_{c_1}, \dots, s_{c_M}) \quad (6)$$

$$\mathbf{S}\mathbf{b} = (s_{b_1}, s_{b_2}, \dots, s_{b_M}) \quad (7)$$

where s_{m_j} , s_{c_j} , and s_{b_j} represent the available memory, computation power, and bandwidth, of the j th server, respectively.

The batch assignment problem can be formulated as a multi-objective optimization problem [26], as follows:

$$\text{minimize } [f_1(\mathbf{X}), f_2(\mathbf{X}), \dots, f_K(\mathbf{X})] \quad (8a)$$

subject to

$$\mathbf{e} \cdot \mathbf{X}^\top = \mathbf{e} \quad (8b)$$

$$\mathbf{m} \cdot \mathbf{X} \preceq \mathbf{S}\mathbf{m} \quad (8c)$$

$$\mathbf{c} \cdot \mathbf{X} \preceq \mathbf{S}\mathbf{c} \quad (8d)$$

$$\mathbf{b} \cdot \mathbf{X} \preceq \mathbf{S}\mathbf{b} \quad (8e)$$

$$x_{ij} \in \{0, 1\} \quad (8f)$$

The cost function $f_i(\mathbf{X})$ is related to the runtime t_i of task i , as follows:

$$f_i(\mathbf{X}) = \alpha \cdot t_i + (1 - \alpha) \cdot T \quad (9)$$

where t_i is the runtime of task i , T is the overall execution time (determined by the container with the longest execution time), and $0 \leq \alpha \leq 1$. The element x_{ij} of matrix $\mathbf{X}$ is a decision variable defined as

$$x_{ij} = \begin{cases} 1 & \text{if task } i \text{ is assigned to server } j \\ 0 & \text{otherwise} \end{cases} \quad (10)$$

Equation (8c)–(8e) depict the server's resource constraints, where $\mathbf{m}$, $\mathbf{c}$, and $\mathbf{b}$ stand for memory, computation, and bandwidth. While it is appealing to define the problem at hand as a minmax (or minimax) problem, we find this presentation unsatisfactory. In minmax problems, one tries to minimize only the maximal value (i.e., the maximal running time in our case). Therefore, in minmax, two solutions with the same maximal value are considered equal, regardless of the values of other (smaller) execution times. Expression (8a) better represents this distinction.

The optimization goal is to minimize the cost function of each task, subject to the servers' resource constraints, while keeping the overall runtime required to complete all tasks as short as possible. Equation (8b) guarantees that any task can be assigned only to a single dedicated server, while Equation (8c)–(8e) ensure that the servers' resources can fulfill the requirements of the overall tasks ($\preceq$ stands for element-wise comparison).

The total bandwidth requirements to communicate between any two servers should be less than the available network bandwidth $\mathbf{Nb}$, as described below:

$$\mathbf{X}^\top \cdot \mathbf{B} \cdot \mathbf{X} \preceq \mathbf{Nb} \quad (11)$$

where $\mathbf{Nb}_{i,j}$ represents the bandwidth on the path for communication from server i to server j .

4. The Proposed Approach

This section presents the multi-agent-based solution using a Deep RL (DRL) approach to cope with the challenges of the container allocation problem. We propose an RL-based multi-agent framework for efficient container placement, using a set of agents in a mixed cooperative–competitive environment. Each agent is provided with a shaped reward and may collaborate with other agents to improve container allocation strategies. The main idea of RL is learning through interaction using a trial-and-error learning approach. An RL agent interacts with its environment and, upon observing the outcomes of its actions, learns to alter its behavior in response to the rewards received [27].

The i th agent observes state $S_i(t)$ from its environment at time step t and interacts with the environment by taking an action a_t in state $S_i(t)$. Based on the current state and the chosen action, the environment and the agent transition to a new state $S_i(t+1)$. Upon each state transition, the environment provides a scalar reward r_{t+1} to the agent as feedback.

The best sequence of actions is determined by the rewards provided by the environment, while the goal of the agent is to maximize the expected return.

An actor–critic framework that considers the action policies of other agents has the potential to successfully learn policies that require dealing with high-complexity multi-agent problems [28,29]. Ryan et al. [28] proposed a simple extension of actor–critic policy gradient methods, where the critic network is provided with extra information about the policies of other agents, while the actor network only has access to local information. In this work, we adopted a similar actor–critic approach to that proposed in [28].

The proposed actor–critic framework is comprised of two types of neural network: an actor network, and a critic network. The actor network is used to generate actions according to the input state and is trained using a gradient-based method. We present a unique training algorithm for actor–critic-based agents.

4.1. Reinforcement Learning Framework

Figure 2 depicts a multi-agent cloud environment. Each agent is assigned a single container and is responsible for deciding on which server to place it. Agents perform the allocation of containers to the servers in a serial predetermined order (first, agent no. 1 allocates its task, while agent K allocates its associated container only at the end of the round). This is achieved by randomly allocating the containers to the various agents. In each round, each agent is responsible for the allocation of a new container. In each round of decisions, a new state is defined for each agent, and another container has the priority of being assigned first. The state defines the container for which the agent is responsible and the previous decisions made by the agents that preceded it in the current round of decisions. In addition, the agent also receives information about the previous decisions of the next agents in line in the same round of decisions. The state also defines the utilization of the servers concerning the previous decision round.

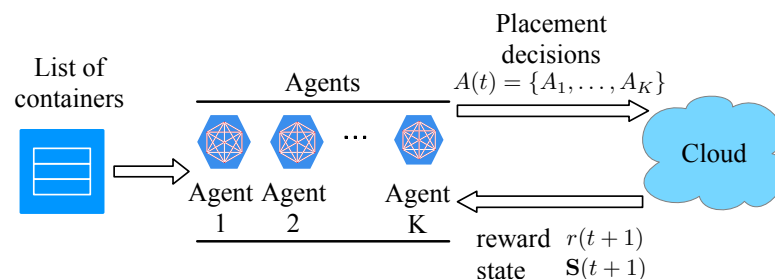


Figure 2. Multi-agent cloud environment.

The following sections formulate the space state, action space, and the reward for the i th agent.

4.1.1. State Space

Equation (12) describes the state space of agent i :

$$S_i(t) = \{\{q_i(t)\}, \{a_1(t), \dots, a_{i-1}(t), a_{i+1}(t-1), \dots, a_K(t-1)\}, \{u_1(t-1), \dots, u_M(t-1)\}\} \quad (12)$$

where $q_i(t)$ denotes the container for which the i th agent is responsible at decision round t the i th agent's task requirement at round t . The previous decisions made by the agents that preceded it in the current round of decisions are denoted by the action vector $a_j(t)$, $j = 1 \dots, i-1$. The previous decisions of the next agents in line in the same round of decisions are represented by the action vector $a_j(t-1)$, $j = i+1, \dots, K$. Finally, $u_j(t)$ stands for the utilization of the servers concerning the previous decision round.

4.1.2. Action Space

All agents share the same action space. Equation (13) describes the common action space

$$A(S) = \{a | a \in \{1, 2, \dots, M\}\} \quad (13)$$

where $\{1, \dots, M\}$ stand for the M available servers.

4.1.3. Reward

Upon each state transition, the environment provides a scalar-shaped reward $r_i(t)$ to the agent as feedback. The reward is comprised of both local and global rewards. Equation (14) describes the reward function:

$$r_i(t) = \alpha \cdot L_i(t) + (1 - \alpha) \cdot G(t) \quad (14)$$

where L_i and G_i denote the local and global rewards of agent i , respectively, and $0 \leq \alpha \leq 1$ represents the relative weights given for each reward. The amount of the reward is decided according to the performance of the specific agent at each round of decisions.

The local reward of the i th agent is represented by the time difference between the overall execution time $g(t)$ (determined by the container with the longest execution time) and the execution time of the container for which the local agent is responsible:

$$L_i(t) = g(t) - t_i(t) \quad (15)$$

where t_i is the runtime of task i .

The global reward is determined relative to the time difference between the overall runtime in the current and previous rounds of decisions:

$$G(t) = g(t) - g(t - 1) \quad (16)$$

4.2. Actor–Critic Approach

The advantage actor–critic (A2C) method has achieved superior performance in solving sequential decision problems [27]. We employed this actor–critic approach to find a policy that maximizes the expected reward R over all possible dialogue trajectories. The actor is responsible for selecting the actions and therefore determining the policy, while the critic estimates the value function and hence criticizes the actor's actions [30].

The policy is a probabilistic mapping function between the state space and the action space and depends on the parameters learned by policy gradient algorithms [30]. Therefore, the policy is directly optimized by gradient descent, concerning the expected return (long-term cumulative reward).

In our proposed multi-agent-based approach, each agent consists of two neural networks: the actor network, and the critic network. We consider two different types of network: LSTM-based agents, and DNC-based agents. Both the LSTM- and the DNC-based networks are implemented using two hidden layers with eight neurons each (representing the number of servers), followed by a fully-connected layer with eight neurons for the actor network and a single neuron for the critic network. The DNC-based network is implemented using the software package provided by [31].

In each decision round, each actor is fed with a different advantage value from its corresponding critic, for each agent. Assuming a set of K actors, the parameters learned by the policy gradient algorithm for each actor are defined by θ_i for the policy π_i (for the i th agent).

Given the objective loss function J , the policy gradient is computed using the advantage function A_i , as follows:

$$\nabla_{\theta_i} J(\theta_i) = \mathbb{E}_{a_i \sim \pi_i} [\nabla_{\theta_i} \log \pi_i(a_i | s_i) A_i(s_i, a_i)] \quad (17)$$

where s_i and a_i represent the state space and the taken action for the i th agent in each decision round. The training of both actor–critic networks is carried out according to Equation (17) applying the gradient-descent algorithm.

4.3. Multi-Agent RL-Based Approach

We propose an efficient model-free multi-agent-based approach that enables the collaboration of multiple agents to solve the container allocation problem in a cloud environment. The interaction between multiple agents in a shared environment can either be competitive or cooperative [12]. In this work, we adopt a mixed cooperative–competitive approach [28]. Each agent aims at minimizing both the execution time of the container assigned to it (i.e., competitive approach) and the overall runtime (i.e., cooperative approach).

The assignment decision for each agent depends on both the local and global rewards, thus considering the decisions taken by all the other agents. In case the container needs to communicate with another container or a remote server, the agent should learn to assign its container to a close or the same host server, to minimize the communication latency.

Since each agent is aware of all assignment decisions taken by the other agents, the environment is stationary even as the decision policy changes, as depicted by [28]:

$$P(s'|s, a_1, \dots, a_K, \pi_1, \dots, \pi_K) = P(s'|s, a_1, \dots, a_K) = P(s'|s, a_1, \dots, a_K, \pi'_1, \dots, \pi'_K) \quad (18)$$

for any $\pi \neq \pi'$.

Foerster et al. [32] suggested a multi-agent actor–critic method called counterfactual multi-agent policy gradients, which uses a centralized critic to estimate the Q -function and decentralized actors to optimize the agents' policies. We adopt a similar approach adapted to both decentralized actors and decentralized critics.

The counterfactual policy estimates the marginal contribution of a single agent's action, while keeping the other agents' actions fixed. The idea of the counterfactual approach is inspired by difference rewards [32]. Difference rewards are a powerful way to perform multi-agent credit assignment, since an agent which directly tries to maximize the system reward has difficulty determining the effect of its actions on its reward [32,33]. In contrast, agents using the difference reward have more influence over the value of their own reward; therefore, a good action taken by the agent is more likely to be reflected in its local reward. Moreover, any action taken by the agent that improves the difference reward also improves the true global reward, which does not depend on the agent's actions. In this work, the impact of the agent's action on the global reward is estimated using the difference between the global and the local agent rewards.

5. Container Allocation Framework

The container placement problem can be considered as batch assigning, where each batch is composed of N tasks, each with different local resource requirements. We propose an RL-based multi-agent framework for efficient container placement, using a set of agents in a mixed cooperative–competitive environment. An RL-based agent interacts with its environment and, upon observing the outcomes of its actions, learns to alter its own behavior in response to the rewards received. Each agent is provided with a shaped reward and may collaborate with other agents to improve container allocation strategies.

Assuming K containers are allocated to M servers, where $K > M$, K agents are required, since each agent is responsible for the placement of a single container. Each container is assigned to one of the given M servers; however, multiple containers can be assigned to the same server. The training of the actor–critic networks for each agent is an iterative process. In each iteration (i.e., decision round), a single allocation per agent is carried out. Then, according to the performance achieved (i.e., the overall execution time and the local reward) a new allocation is generated in the next round. The training process is terminated when the contribution to the execution time improvement is marginal.

Agents perform the allocation of containers to the servers in a fixed serial predetermined order, while each agent is assigned a single container and is responsible for deciding on

which server to place it. The containers are randomly allocated to the various available agents. In each decision round, each agent is coupled with a different container; thus, another container receives the priority for being assigned. Each agent is informed of the previous decisions made by the preceding agents and of the servers' utilization concerning the previous decision round.

Algorithm 1 depicts the container allocation process. The allocation process is carried out iteratively (lines 2–14) by training the actor and critic networks for each agent. In each iteration, the containers are randomly assigned to the agents (line 3), and the shaped reward is calculated according to the performance of each agent (lines 8–10). Then the critic estimates the advantage value (line 11) and, finally, the critic and actor networks weights are updated (lines 12–13).

Algorithm 1 Container Allocation Process.

Input: K containers, M servers
Output: Optimal containers assignment (i.e., minimal overall execution time)

- 1 Initialize random parameters: actor networks weights $\{\theta_1, \dots, \theta_K\}$, critic networks weights $\{\delta_1, \dots, \delta_K\}$
- 2 **for** $t = 1, \dots, N$ **do** // *decision_round* = t
- 3 Containers are randomly allocated to the agents;
- 4 Observe current state space $\mathbf{s}(t-1)$;
- 5 Each agent $i \in \{1, \dots, K\}$ selects action in a sequential order,
 $a^i(t) = \{a^i | a^i \in \{1, 2, \dots, M\}\}$ and the state space is updated;
- 6 A new state $\mathbf{s}(t)$ is generated;
- 7 A single container allocation per agent is carried out;
- 8 The overall execution time and the execution time of each container are measured;
- 9 If the improvement in the overall execution time is less than Th , then **exit**
- 10 The shaped rewards $\mathbf{r} = \{r_1(t), \dots, r_K(t)\}$ are calculated and fed to the critics networks;
- 11 The critic networks estimate the advantage value function for each agent:
 $A_i(s_i, a_i) = r_i(t) + \delta V_i(s_i(t)) - V_i(s_i(t-1))$;
- 12 Update critics networks weights ;
- 13 Update actors networks weights according to
 $\nabla_{\theta_i} J(\theta_i) = \mathbb{E}_{a_i \sim \pi_i} [\nabla_{\theta_i} \log \pi_i(a_i | s_i) A_i(s_i, a_i)]$;
- 14 **end**
- 15 **Return:** trained actors and critics networks

5.1. Complexity

The complexity of the state–action space, and consequently the DRL complexity, is proportional to the total number of agents in the system. Y. Ren et al. [34] provided a detailed complexity analysis for a similar multi-agent DRL-based approach applied to fog computing, offloading computation-intensive tasks to fog access points. The training process runs offline and is performed in the cloud that has sufficient computation resources. Hence, we mainly pay attention to the complexity of online decision making.

In our multi-agent DRL approach, each task is assigned to a DRL-based agent. Hence, the total complexity relies on the complexity of a single DRL-based agent. Based on the complexity analysis described in [34], we can formulate the total complexity of our DRL-based approach as follows:

$$\mathcal{O}(2KH(KM + LH)) \quad (19)$$

where K is the number of containers, M is the numbers of servers, and assuming L hidden layers with H neurons in each hidden layer.

5.2. Existing Container Allocation Approaches

We compared the performance of the LSTM and the DNC multi-agents against the well-known bin packing heuristics and the common open-source Kubernetes allocation tool used for allocation, deployment, and management of containerized applications [35,36]. As a deployment tool, Kubernetes handles container distribution using a heuristic bin-packing algorithm. An integrated solution for the proposed container placement approach with the Kubernetes orchestration tool is provided by replacing the Kubernetes decision mechanism with the allocation deployment suggested in the proposed RL multi-agent approach.

As carried out by Kubernetes, two scoring strategies are evaluated in this work, to support the bin packing of resources: (a) best-fit and (b) max-fit bin-packing algorithms. The best-fit strategy allows specifying the resources along with weights for each resource, to score servers based on the request-to-capacity ratio [33]. In this case, a scheduled container is assigned to the server with the lowest available resource capacity (which can still accommodate the container) [15]. On the other hand, the max-fit strategy scores the servers based on the utilization of resources, favoring the ones with higher allocation. Therefore, a scheduled container is assigned to the server with the highest available resource capacity. We apply a 3D bin packing allocation for both the max-fit and best-fit algorithms [37].

6. Experimental Results

This section presents the results of the proposed multi-agent RL-based approach. The achieved performance is compared with the common Kubernetes tool and classical bin-packing heuristic algorithms. The results showed that the proposed approach outperformed the Kubernetes and the greedy bin-packing algorithm for LSTM- and DNC-based agents. The implementation of both agents was carried out using the Tensor Flow machine learning platform and written in Python.

It is important to emphasize that the experimental results were carried out in real time using a real private cloud environment. The experimental environment included eight servers, which were deployed as virtual machines. The servers were characterized by different processors, memory, and CPU resources, as depicted in Table 1. The deployment of the containers was carried out using the Kubernetes tool. Although the implementation in the present work is considered an “on-premise private cloud”, the same approach may be used for public and/or hybrid clouds in which Kubernetes is usually used for orchestrating the deployment of containers.

Table 1. Computation resources.

Resource	S1	S2	S3	S4	S5	S6	S7	S8
CPU	xeon		i5 8500		i5 2400			
Cores	4	4	3	3	2	2	2	2
RAM [GB]	4	3	4	3	4	3	4	3

Figure 3 depicts the overall batch execution time as a function of the number of iterations, for both LSTM-based and DNC-based agents. The training process converges after about 350 iterations.

For the DNC-based agents, the best result was achieved with a learning rate (LR) of 0.04 and a memory matrix size of 17×17 . The LSTM-based agents outperformed the DNC agents and resulted in a faster batch execution time.

Both the training set and the testing set included various task batches, where each batch included up to 28 different tasks. Each task was characterized by different resource requirements (i.e., CPU time, memory, and communication bandwidth). For the testing, a total of 30 different task batches were used, and the average results are presented.

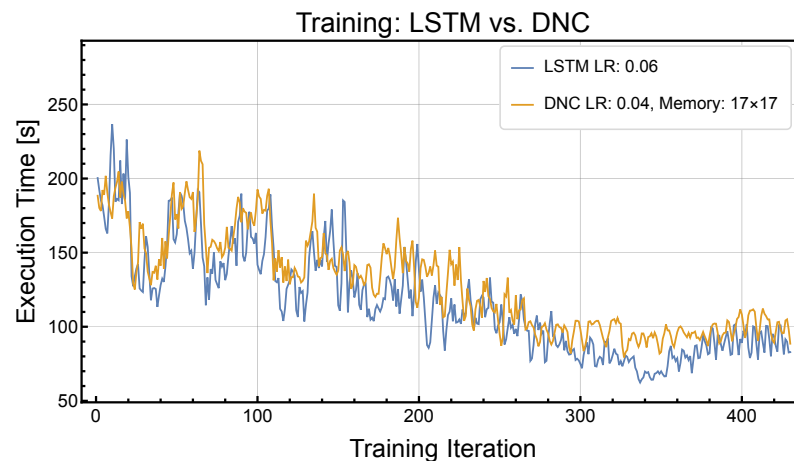


Figure 3. Batch execution time for LSTM and DNC agents.

Figure 4 compares the performance of the proposed LSTM and DNC agents to the Kubernetes tool and the two classical bin-packing algorithms. For the testing set, the results show that the proposed RL-based approach outperformed the three heuristic classical approaches in terms of the average batch execution time.

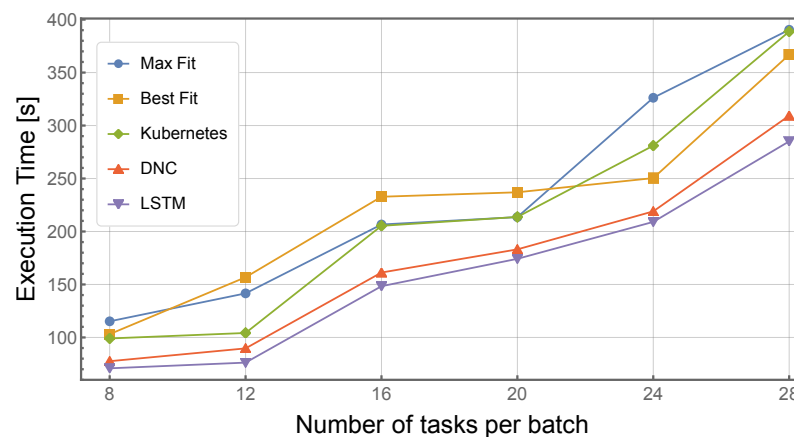


Figure 4. Batch execution time vs. system load.

For all scenarios with less than 20 tasks per batch, the Kubernetes tool showed the best results, compared to the other classical methods.

The LSTM achieved the best performance for all tested scenarios, demonstrating an average improvement of about 25%, 31%, and 32% compared to the Kubernetes, Best-Fit, and Max-Fit, correspondingly. The LSTM execution time ranged from 70 to 285 s, for 8 and 28 tasks per batch, respectively, while the DNC-based agents resulted in a slower execution time of about 8%.

Table 2 depicts the average completion time achieved for the mixed tasks, i.e., tasks that included communication. The results show an improvement of 26% and 30% for 8 and 12 tasks per batch when using the proposed LSTM compared to the best-performing bin packing method. For the scenarios with batches that included more than 12 tasks, the average improvement reduced to about 10%, which means that as the number of tasks per batch increased, the classical methods performed better.

Figure 5 shows the average execution times for local tasks (which do not require communication with other tasks) as a function of the number of tasks. The superiority of the proposed approach is not decisive. With up to 16 tasks, the proposed RL-based approach showed the best results, while with over 16 tasks the max-fit approach seems preferable. The LSTM- and DNC-based agents demonstrated similar results to the Kubernetes. The best-fit method demonstrated the worst performance for all cases.

Table 2. Average completion time.

No. of Tasks	Max Fit	Best Fit	Kuber.	DNC	LSTM	LSTM Improv.
8	99	86	82	66	61	26%
12	115	126	93	70	65	30%
16	169	179	153	148	135	12%
20	178	191	181	165	158	11%
24	254	206	225	193	182	11%
28	303	272	286	269	249	9%

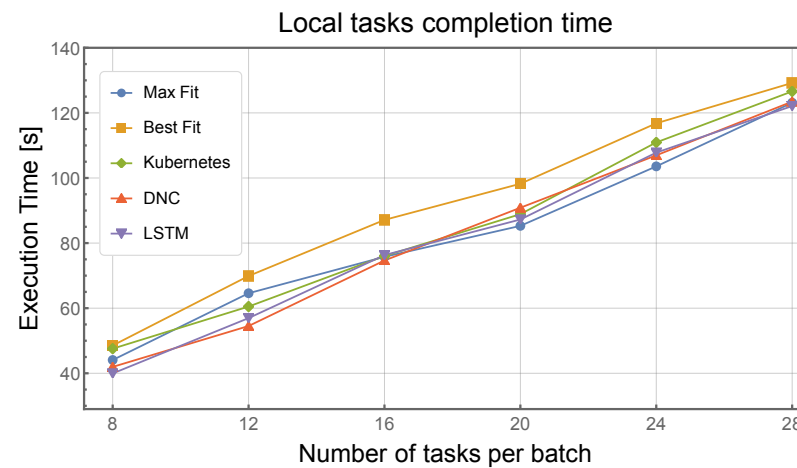
**Figure 5.** Average completion time for local tasks.

Figure 6 shows the average completion times for tasks that included communication requirements. The proposed RL-based approach outperformed the three classical approaches in terms of the average batch execution time. The results showed that the LSTM-based agents outperformed the DNC-based agents, while the Kubernetes outperformed the max-fit and best-fit approaches (up to 20 tasks).

From the results shown in Figures 5 and 6, it follows that the relative advantage of the proposed RL-based approach was mainly due to the efficiency in allocating tasks that included communication with other tasks.

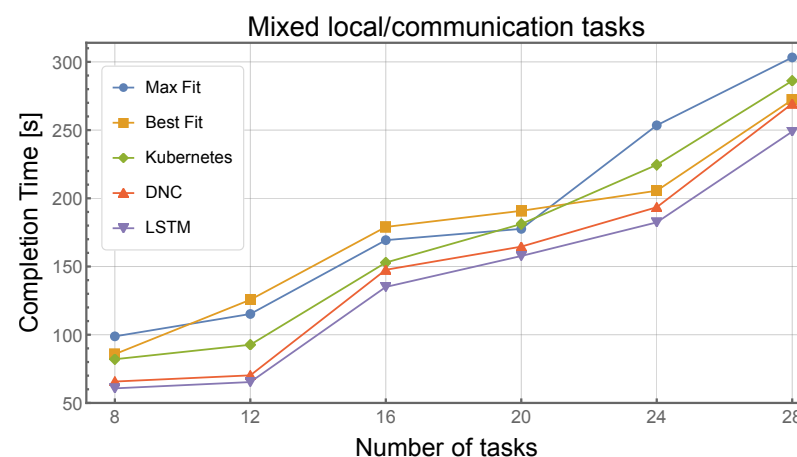
**Figure 6.** Average completion time for mixed tasks.

Figure 7 depicts the performance of the five different approaches as a function of system load (i.e., number of tasks), in terms of statistic parameters. The DRL-based agents outperformed the classical approaches, while the LSTM-based agent demonstrated the best results for all scenarios. Figure 7 shows that the efficiency of the DRL-based agents significantly increased as the system load increased. For example, the median batch execution time for

28 tasks was 285 s and 394 s for the LSTM and Kubernetes agents, respectively. The results indicate that our DRL-based approach achieved a 30% speedup, compared to the common Kubernetes approach.

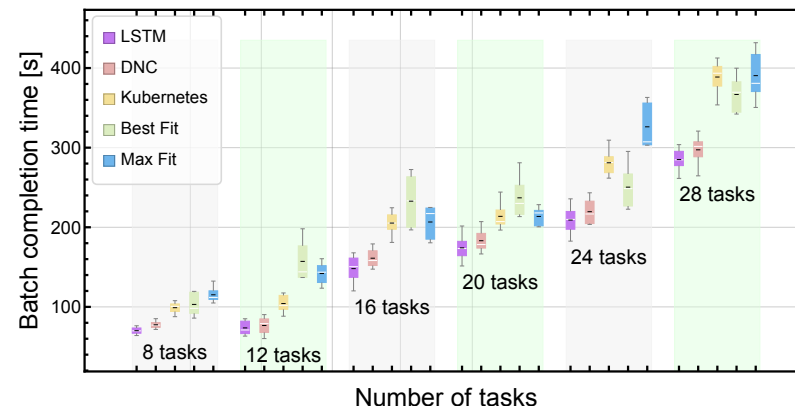


Figure 7. Performance vs. system load.

Since the performance criteria were chosen as the completion time of the slowest task, we evaluated the effect of the allocation method on the remaining tasks. Let us consider that the batch completion time is T . Figure 7 shows that the completion time of 25% of the tasks was close to T , within a time range of 2.5%, 3%, 3.5% for the Kubernetes, LSTM, and best-fit approaches, respectively. Moreover, 75% of the tasks ended within a time range of 4.5%, 6%, and 12% of T , for the Kubernetes, LSTM and best-fit approaches, respectively. Therefore, the chosen completion time criterion, practically represents the average completion time of the tasks and the proposed allocation methods fairly distributed the tasks among the servers. Figure 8 depicts the distribution of the average tasks among the 8 servers for a batch of 16 tasks.

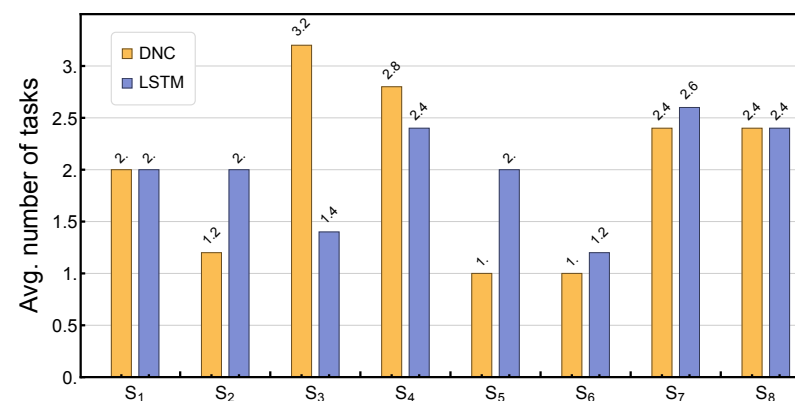


Figure 8. Task distribution among the servers (for 16 tasks).

7. Summary and Conclusions

This paper presents a multi-agent-based framework using a deep reinforcement learning-based approach to cope with the challenges of the container allocation problem. This work suggests a new approach for assigning containers to servers in a cloud environment, while meeting computing resource requirements and minimizing the overall task completion time. We showed that decentralizing the allocation decisions among multiple agents effectively tackles the high complexity of task allocation in a cloud environment. Two RL-based agents, the LSTM-based and DNC-based agents, were evaluated, demonstrating their efficiency compared to the well-known bin packing heuristics and the common Kubernetes allocation orchestration tool. The experimental results showed that both DRL-based approaches were superior in terms of the overall runtime and demonstrated an improvement of about 28% compared to the existing techniques. Furthermore,

the results showed that the LSTM-based agents outperformed the DNC-based agents, while the Kubernetes outperformed the max-fit and best-fit approaches. We conclude that the relative advantage of the proposed RL-based approach is mainly due to its efficiency in allocating tasks that include communication with other tasks.

Author Contributions: Conceptualization, all authors; methodology, all authors; software, T.D.; validation, all authors; formal analysis, all authors; investigation, all authors; resources, Y.B.-S. and S.G.; writing—original draft preparation, all authors; writing—review and editing, all authors; visualization, T.D. and Y.B.-S.; supervision, Y.B.-S. and S.G.; project administration, Y.B.-S. and S.G. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Conflicts of Interest: The authors declare no conflict of interest.

References

- Joy, A.M. Performance comparison between Linux containers and virtual machines. In Proceedings of the 2015 International Conference on Advances in Computer Engineering and Applications, Ghaziabad, India, 19–20 March 2015; pp. 342–346. [CrossRef]
- Vmware. Containers Deployment. Available online: <https://www.vmware.com/topics/glossary/content/container-deployment> (accessed on 1 April 2023).
- Kubernetes. Pod Lifecycle. Available online: <https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/> (accessed on 1 April 2023).
- Rossi, F.; Cardellini, V.; Lo Presti, F.; Nardelli, M. Geo-distributed efficient deployment of containers with Kubernetes. *Comput. Commun.* **2020**, *159*, 161–174. [CrossRef]
- Rossi, F.; Nardelli, M.; Cardellini, V. Horizontal and Vertical Scaling of Container-Based Applications Using Reinforcement Learning. In Proceedings of the 2019 IEEE 12th International Conference on Cloud Computing (CLOUD), Milan, Italy, 8–13 July 2019; pp. 329–338. [CrossRef]
- Liu, N.; Li, Z.; Xu, J.; Xu, Z.; Lin, S.; Qiu, Q.; Tang, J.; Wang, Y. A hierarchical framework of cloud resource allocation and power management using deep reinforcement learning. In Proceedings of the 2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS), Atlanta, GA, USA, 2–8 June 2017; IEEE: Piscataway, NJ, USA, 2017; pp. 372–382.
- Ju, Y.; Chen, Y.; Cao, Z.; Liu, L.; Pei, Q.; Xiao, M.; Ota, K.; Dong, M.; Leung, V.C.M. Joint Secure Offloading and Resource Allocation for Vehicular Edge Computing Network: A Multi-Agent Deep Reinforcement Learning Approach. *IEEE Trans. Intell. Transp. Syst.* **2023**, *24*, 5555–5569. [CrossRef]
- Narantuya, J.; Shin, J.S.; Park, S.; Kim, J. Multi-Agent Deep Reinforcement Learning-Based Resource Allocation in HPC/AI Converged Cluster. *Comput. Mater. Contin.* **2022**, *72*, 4375–4395. [CrossRef]
- Suzuki, A.; Kobayashi, M.; Oki, E. Multi-Agent Deep Reinforcement Learning for Cooperative Computing Offloading and Route Optimization in Multi Cloud-Edge Networks. *IEEE Trans. Netw. Serv. Manag.* **2023**. [CrossRef]
- Rossi, F. Auto-scaling Policies to Adapt the Application Deployment in Kubernetes. In Proceedings of the 12th ZEUS Workshop 2020 (ZEUS 2020), Potsdam, Germany, 20–21 February 2020.
- Ningcheng Yuan, C.J. A DRL-Based Container Placement Scheme with Auxiliary Tasks. *Comput. Mater. Contin.* **2020**, *64*, 1657–1671. [CrossRef]
- Busoniu, L.; Babuska, R.; De Schutter, B. A Comprehensive Survey of Multiagent Reinforcement Learning. *IEEE Trans. Syst. Man, Cybern. Part C (Appl. Rev.)* **2008**, *38*, 156–172. [CrossRef]
- Alwarafy, A.; Çiftler, B.S.; Abdallah, M.; Hamdi, M.; Al-Dhahir, N. Hierarchical Multi-Agent DRL-Based Framework for Joint Multi-RAT Assignment and Dynamic Resource Allocation in Next-Generation HetNets. *IEEE Trans. Netw. Sci. Eng.* **2022**, *9*, 2481–2494. [CrossRef]
- Horovitz, S.; Arian, Y. Efficient Cloud Auto-Scaling with SLA Objective Using Q-Learning. In Proceedings of the 2018 IEEE 6th International Conference on Future Internet of Things and Cloud (FiCloud), Barcelona, Spain, 6–8 August 2018; pp. 85–92. [CrossRef]
- Hussein, M.; Mousa, M.; Alqarni, M. A placement architecture for a container as a service (CaaS) in a cloud environment. *J. Cloud Comput.* **2019**, *8*, 7. [CrossRef]
- Zhang, R.; Zhong, A.m.; Dong, B.; Tian, F.; Li, R.; Zhang, L.J. Container-VM-PM Architecture: A Novel Architecture for Docker Container Placement. In *Cloud Computing—CLOUD 2018: Proceedings of the 11th International Conference, Held as Part of the Services Conference Federation, SCF 2018, Seattle, WA, USA, 25–30 June 2018*; Springer International Publishing: Cham, Switzerland, 2018; pp. 128–140.
- Mao, Y.; Oak, J.; Pompili, A.; Beer, D.; Han, T.; Hu, P. DRAPS: Dynamic and resource-aware placement scheme for docker containers in a heterogeneous cluster. In Proceedings of the 2017 IEEE 36th International Performance Computing and Communications Conference (IPCCC), San Diego, CA, USA, 10–12 December 2017; pp. 1–8. [CrossRef]
- Guan, X.; Wan, X.; Choi, B.Y.; Song, S.; Zhu, J. Application Oriented Dynamic Resource Allocation for Data Centers Using Docker Containers. *IEEE Commun. Lett.* **2017**, *21*, 504–507. [CrossRef]

19. Usmani, Z.; Singh, S. A survey of virtual machine placement techniques in a cloud data center. *Procedia Comput. Sci.* **2016**, *78*, 491–498. [\[CrossRef\]](#)
20. Abrishami, S.; Naghibzadeh, M.; Epema, D.H. Deadline-constrained workflow scheduling algorithms for Infrastructure as a Service Clouds. *Future Gener. Comput. Syst.* **2013**, *29*, 158–169. [: 10.1016/j.future.2012.05.004. \[CrossRef\]](#)
21. Li, X.; Cai, Z. Elastic Resource Provisioning for Cloud Workflow Applications. *IEEE Trans. Autom. Sci. Eng.* **2017**, *14*, 1195–1210. [\[CrossRef\]](#)
22. Cai, Z.; Li, X.; Ruiz, R. Resource Provisioning for Task-Batch Based Workflows with Deadlines in Public Clouds. *IEEE Trans. Cloud Comput.* **2019**, *7*, 814–826. [\[CrossRef\]](#)
23. Chen, Y.; Wang, L.; Chen, X.; Ranjan, R.; Zomaya, A.Y.; Zhou, Y.; Hu, S. Stochastic Workload Scheduling for Uncoordinated Datacenter Clouds with Multiple QoS Constraints. *IEEE Trans. Cloud Comput.* **2020**, *8*, 1284–1295. [\[CrossRef\]](#)
24. Graves, A.; Wayne, G.; Reynolds, M.; Harley, T.; Danihelka, I.; Grabska-Barwinska, A.; Colmenarejo, S.G.; Grefenstette, E.; Ramalho, T.; Agapiou, J.; et al. Hybrid computing using a neural network with dynamic external memory. *Nature* **2016**, *538*, 471–476. [\[CrossRef\]](#) [\[PubMed\]](#)
25. Nasir, Y.S.; Guo, D. Multi-Agent Deep Reinforcement Learning for Dynamic Power Allocation in Wireless Networks. *IEEE J. Sel. Areas Commun.* **2019**, *37*, 2239–2250. [\[CrossRef\]](#)
26. Awad, M.; Khanna, R. *Efficient Learning Machines: Theories, Concepts, and Applications for Engineers and System Designers*; Springer Nature: Berlin/Heidelberg, Germany, 2015.
27. Arulkumaran, K.; Deisenroth, M.P.; Brundage, M.; Bharath, A.A. Deep Reinforcement Learning: A Brief Survey. *IEEE Signal Process. Mag.* **2017**, *34*, 26–38. [\[CrossRef\]](#)
28. Lowe, R.; Wu, Y.; Tamar, A.; Harb, J.; Abbeel, P.; Mordatch, I. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. In Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17), Long Beach, CA, USA, 4–9 December 2017; Curran Associates Inc.: Red Hook, NY, USA, 2017; pp. 6382–6393.
29. Sun, P.; Guo, Z.; Liu, S.; Lan, J.; Wang, J.; Hu, Y. SmartFCT: Improving power-efficiency for data center networks with deep reinforcement learning. *Comput. Netw.* **2020**, *179*, 107255. [: 10.1016/j.comnet.2020.107255. \[CrossRef\]](#)
30. Peng, B.; Li, X.; Gao, J.; Liu, J.; Chen, Y.N.; Wong, K.F. Adversarial advantage actor-critic model for task-completion dialogue policy learning. In Proceedings of the 2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Calgary, AB, Canada, 15–20 April 2018; IEEE: Piscataway, NJ, USA, 2018; pp. 6149–6153.
31. Deep Mind. DNC Implementation Github. Available online: <https://github.com/deepmind/dnc> (accessed on 1 April 2023).
32. Foerster, J.N.; Farquhar, G.; Afouras, T.; Nardelli, N.; Whiteson, S. Counterfactual Multi-Agent Policy Gradients. In Proceedings of the AAAI, New Orleans, LA, USA, 2–7 February 2018.
33. Tumer, K.; Agogino, A. Distributed agent-based air traffic flow management. In Proceedings of the 6th International Joint Conference on Autonomous Agents and Multiagent Systems, Honolulu, HI, USA, 14–18 May 2007; pp. 1–8.
34. Ren, Y.; Sun, Y.; Peng, M. Deep Reinforcement Learning Based Computation Offloading in Fog Enabled Industrial Internet of Things. *IEEE Trans. Ind. Inform.* **2021**, *17*, 4978–4987. [\[CrossRef\]](#)
35. Google Kubernetes. What Is Kubernetes. Available online: <https://kubernetes.io/docs/concepts/overview/what-is-kubernetes/> (accessed on 1 April 2023).
36. Google Kubernetes. Scheduling Framework. Available online: <https://kubernetes.io/docs/concepts/scheduling-eviction/scheduling-framework/#normalize-scoring> (accessed on 1 April 2023).
37. Dube, E. Optimizing Three-Dimensional Bin Packing Through Simulation. In Proceedings of the Sixth IASTED International Conference Modelling, Simulation, and Optimization, Gaborone, Botswana, 11–13 September 2006.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.